CRISP-DM Linear Regression

write skelearn to solve linear regression problem, following crisp-dm workflow

Here is a simple **scikit-learn linear regression** example following the **CRISP-DM workflow**.

Python
```python
# CRISP-DM Linear Regression Example using scikit-learn

# 1. Business Understanding
# Goal: Predict house price based on features such as area, bedrooms, and age.

# 2. Data Understanding
import pandas as pd

data = pd.DataFrame({
    "area": [50, 60, 80, 100, 120, 150],
    "bedrooms": [1, 2, 2, 3, 3, 4],
    "age": [10, 8, 6, 5, 3, 1],
    "price": [300, 350, 450, 600, 700, 900]
})

print(data.head())
print(data.info())
print(data.describe())

# 3. Data Preparation
from sklearn.model_selection import train_test_split

X = data[["area", "bedrooms", "age"]]
y = data["price"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# 4. Modeling
from sklearn.linear_model import LinearRegression

model = LinearRegression()
model.fit(X_train, y_train)

# 5. Evaluation
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
import numpy as np

y_pred = model.predict(X_test)

mae = mean_absolute_error(y_test, y_pred)
rmse = np.sqrt(mean_squared_error(y_test, y_pred))
r2 = r2_score(y_test, y_pred)

print("MAE:", mae)
print("RMSE:", rmse)
print("R² Score:", r2)

# 6. Deployment
new_house = pd.DataFrame({
    "area": [90],
    "bedrooms": [3],
    "age": [4]
})

predicted_price = model.predict(new_house)

print("Predicted price:", predicted_price[0])
```

Note: it should be `sklearn`, not `skelearn`.

> list top 10 tasks for data preprocessing

Here are **10 essential data preprocessing tasks**, organized in a practical order (close to how you'd follow them in a real pipeline like CRISP-DM):

---

1. Data Cleaning

Fix obvious issues:

- Remove duplicates
- Correct inconsistent labels (e.g., "Male" vs "male")
- Fix data entry errors

---

2. Handling Missing Values

Decide how to deal with nulls:

- Drop rows/columns
- Impute (mean, median, mode, or model-based)

---

3. Data Type Conversion

Ensure correct formats:

- Convert strings → numeric/datetime
- Ensure categorical vs numerical types are correct

---

4. Handling Outliers

Detect and treat extreme values:

- IQR method
- Z-score
- Capping (winsorization)

---

5. Feature Scaling

Normalize numeric features:

- Standardization (Z-score)

- Min-Max scaling
- Important for models like regression, SVM, KNN

---

6. Encoding Categorical Variables

Convert categories into numbers:

- One-hot encoding
- Label encoding
- Target encoding (advanced)

---

7. Feature Engineering

Create better features:

- Combine variables (e.g., price per area)
- Extract from datetime (year, month, weekday)
- Domain-specific transformations

---

8. Feature Selection

Keep only useful variables:

- Remove irrelevant features
- Correlation analysis
- Use model-based methods (e.g., Lasso, tree importance)

---

9. Handling Imbalanced Data

For classification problems:

- Oversampling (SMOTE)
- Undersampling
- Class weighting

---

10. Data Splitting

Prepare for modeling:

- Train / validation / test split
- Cross-validation setup

Quick Insight

If you're doing **machine learning projects (like your linear regression earlier)**, the most critical ones are:

👉 Missing values
👉 Scaling
👉 Encoding
👉 Feature selection

If you want, I can give you a **real sklearn preprocessing pipeline code** that combines these steps cleanly (very useful for projects + interviews).

> give me best prompt to:
>
> write sklearn to solve regression problem, following crisp-dm random generate n points(x,y), n from 100 to 1000, y=ax+b+noise, a is from -10 to 10, noise is N(mean, var)var is from 0 to 1000

Here's a **clean, high-quality prompt** you can use (for ChatGPT / coding LLMs) to get a strong, structured answer:

✅ Best Prompt

> Write a complete Python program using the **scikit-learn (`sklearn`)** library to solve a **linear regression problem**, strictly following the **CRISP-DM workflow**.
>
> Requirements:
>
> 1. **Business Understanding**
> - Briefly explain the goal of predicting y from x.
> 2. **Data Understanding**
> - Randomly generate a dataset with:
> - n data points, where $n \in [100, 1000]$
> - $x \sim \text{Uniform}(0, 100)$
> - $a \sim \text{Uniform}(-10, 10)$
> - $b \sim \text{Uniform}(-50, 50)$
> - noise $\sim \mathcal{N}(\text{mean} = 0, \text{variance} \in [0, 1000])$
> - $y = ax + b + noise$
> - Print basic statistics and visualize the data.
> 3. **Data Preparation**
> - Split the dataset into training and testing sets
> - (Optional) Apply scaling if necessary

4. **Modeling**
    - Use `sklearn.linear_model.LinearRegression`
    - Train the model on the training data
5. **Evaluation**
    - Evaluate using:
        - Mean Squared Error (MSE)
        - R^2 score
    - Compare predicted vs actual values
6. **Deployment**
    - Predict y for new unseen x values

Additional Requirements:

- Use clear comments for each CRISP-DM phase
- Use `numpy`, `pandas`, `matplotlib` for data handling and visualization
- Ensure reproducibility using a fixed random seed
- Keep the code clean, modular, and well-structured

Output:

- Full runnable Python code
- Include plots of:
    - Raw data
    - Regression line vs actual data

🔥 Why this prompt is strong

- 明確限制數學分布（很多人會寫錯 noise）
- 強制 CRISP-DM 結構（老師很愛）
- 要求 visualization（加分）
- 要 reproducibility（專業）

如果你要，我可以**直接幫你生成滿分版本程式碼（含圖 + 最佳寫法）**。

> we want to deploy on streamlit.app

Use this improved prompt:

```
Write a complete Streamlit app for deployment on streamlit.app that uses sklearn to solve a linear
regression problem, following the CRISP-DM workflow.
```

```
Requirements:

1. Business Understanding
- Explain the goal: predict y from x using linear regression.

2. Data Understanding
- Randomly generate n points, where n is controlled by a Streamlit slider from 100 to 1000.
- Generate:
  x ~ Uniform(0, 100)
  a ~ Uniform(-10, 10)
  b ~ Uniform(-50, 50)
  noise ~ Normal(mean, variance)
  variance is controlled by a Streamlit slider from 0 to 1000
  y = ax + b + noise
- Show the true values of a, b, and noise variance.
- Display the dataset preview and summary statistics.

3. Data Preparation
- Split the data into training and testing sets.
- Allow the user to choose the test size using a Streamlit slider.

4. Modeling
- Use sklearn.linear_model.LinearRegression.
- Train the model on the generated data.

5. Evaluation
- Display:
  Mean Squared Error
  Root Mean Squared Error
  Mean Absolute Error
  R² score
- Show learned slope and intercept.

6. Deployment
- Let users input a new x value using a Streamlit number input.
- Predict the corresponding y value.
- Display the prediction clearly.

Visualization:
- Use matplotlib or plotly.
- Show:
  raw generated data
  train/test points
  regression line
  predicted point for user input

Technical requirements:
- The app must run on streamlit.app.
- Provide the complete app.py code.
- Provide requirements.txt.
- Use numpy, pandas, scikit-learn, streamlit, and matplotlib or plotly.
- Use a random seed input for reproducibility.
- Organize the app with clear headings for each CRISP-DM phase.
- Keep the code clean, beginner-friendly, and well-commented.
```

For **`requirements.txt`**:

```
streamlit
numpy
pandas
scikit-learn
matplotlib
```

i need 100 words only

Write a complete **Streamlit app** using **scikit-learn** to solve a linear regression problem following CRISP-DM. Generate n points (100–1000) with x ~ Uniform(0,100), a ~ Uniform(-10,10), b ~ Uniform(-50,50), noise ~ Normal(0, variance), variance in [0,1000], and y = ax + b + noise. Include sliders for n, variance, and test size, and a seed for reproducibility. Split data, train LinearRegression, evaluate with MSE, RMSE, MAE, R^2, and display coefficients. Visualize data and regression line. Allow user input x for prediction. Provide complete app.py and requirements.txt. Include comments for each CRISP-DM phase.